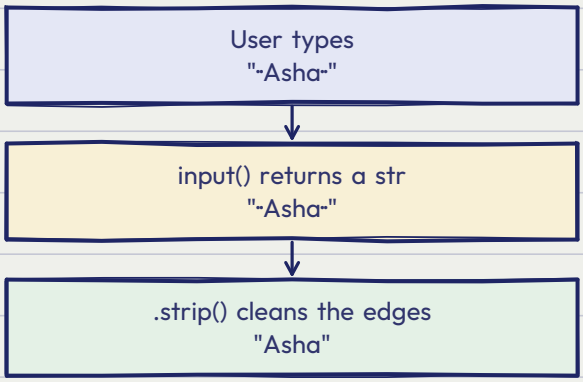
READING INPUT & STRIP()

`input()` returns **text**; `.strip()` removes spaces from the **start and end**.

1) What input() does

- **Pauses** the program and shows a prompt.
- Returns what the user types **as text** (a `str`).
- Python is **case-sensitive**: write `input()`, not `Input()`.

How the data travels



2) Read a name and clean it

Example :

```
name = input("Enter a name: ").strip()
print("Welcome,", name)
```

Output :

```
Enter a name:    Asha
Welcome, Asha
```

the spaces typed around Asha are gone

3) The strip family

Method	What it trims	Result on "-Python class-"
`strip()`	both ends	`"Python class"`
`lstrip()`	left side only	`"Python class··"`
`rstrip()`	right side only	`"··Python class"`

(· stands for a space)

☆ REMEMBER

- `strip()` is a string **method**, so the parentheses matter: `input("Name: ").strip()`.
- `strip()` only trims the **ends**. Spaces between words stay.

Predict first

What does `"·hi·".lstrip()` return? **"hi·"** (the space on the right stays)

Quick check

`"·Python class·".strip()` returns **"Python class"**

When a user types 25, `type(input())` is **<class 'str'>**

Type Mismatch & Conversion

Typing digits does not make a number. The value arrives as **text**.

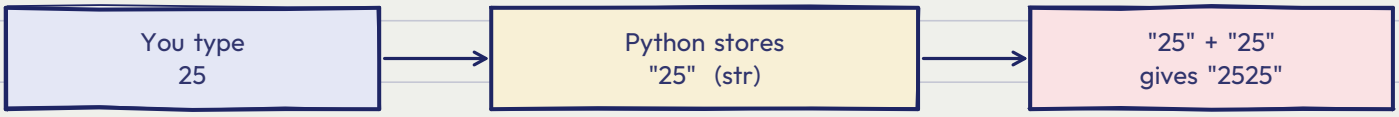
1) Data type mismatch

Example :

```
number = input("Enter your number: ")
print(f"Hello {number}", type(number))
```

Output :

```
Enter your number: 25
Hello 25 <class 'str'>
```



Predict first

If `number = "25"`, then `number + number` gives the **text** `"2525"` (joined), not the number 50.

2) Convert with int() and float()

Call	What it does	Result
`int("25")`	text to whole number	25
`float("25.5")`	text to decimal	25.5
`int("25.5")`	not an integer literal	ValueError
`int(float("25.5"))`	decimal, then truncate	25

Example :

```
age = int(input("Enter your age: "))
print("Next year you will be", age + 1)

price = float(input("Enter item price: "))
quantity = int(input("Enter quantity: "))
total = price * quantity
print("Total:", total)
```

Output :

```
Enter your age: 19
Next year you will be 20
Enter item price: 49.5
Enter quantity: 3
Total: 148.5
```

☆ REMEMBER

- Whole numbers: `int()`. Decimals: `float()`.
- Convert **before** calculating, because `input()` always gives text.

Quick check

What happens when Python runs `int("25.5")`? **It raises a ValueError**

A user types 25 into `input()`. What is the type? **<class 'str'>**

Safe Input & Print()

Handle bad input with **try / except**, then shape output with **sep** and **end**.

1) Handle invalid numbers

- **try**: run the risky conversion.
- **except ValueError**: runs only if the text is not a number.
- **else**: runs when there was no error.

Example :

```
age_text = input("Enter your age: ").strip()

try:
    age = int(age_text)
except ValueError:
    print("Please enter a whole number, such as 18.")
else:
    print(f"Next year you will be {age + 1}.")
```

Output :

```
Enter your age: abc
Please enter a whole number, such as 18.

Enter your age: 18
Next year you will be 19.
```

☆ TRY IT

- To accept a decimal age such as 18.5, swap `int()` for `float()`.

2) Control output with print()

Option	What it controls	Default
sep=	text placed **between** the items	one space
end=	text printed **at the end**	new line (\n)

Example :

```
print("Python", "Basics", "Day 2")
print("2026", "10", "13", sep="-")
print("Loading", end="...")
print("Done!")
```

Output :

```
Python Basics Day 2
2026-10-13
Loading...Done!
```

Predict first

`print("a", "b", "c", sep="")` prints **abc** (nothing between the items)

Quick check

`print("2026", "10", "13", sep="-")` displays **2026-10-13**

F-STRINGS & NUMBER FORMATS

An **f-string** puts variables inside **{ }** and formats numbers with a colon.

1) f-string basics

Example :

```
name = "Asha"
age = 19
marks = 87.456
print(f"{name} is {age} years old.")
print(f"Marks: {marks:.2f}")
print(f"{name} scored {marks:.1f} marks.")
```

Output :

```
Asha is 19 years old.
Marks: 87.46
Asha scored 87.5 marks.
```

2) Number formats

Format	Meaning	Result
`{marks:.2f}`	2 decimal places (rounded)	87.456 gives 87.46
`{marks:.1f}`	1 decimal place	87.456 gives 87.5
`{pct:.1%}`	multiply by 100, add %	0.875 gives 87.5%
`{a + b}`	any expression works	2 + 3 gives 5

3) Three ways to build one message

Style	Code	Note
+ and str()	`name + " is " + str(age) + " years old."`	needs str()
commas	`print(name, "is", age, "years old.")`	spaces added
f-string	`print(f"{name} is {age} years old.")`	clearest

4) The older .format() style

Example :

```
name = "Ravi"
marks = 91.5
print("{} scored {} marks.".format(name, marks))
print("{:.2f}".format(marks))
```

Output :

```
Ravi scored 91.5 marks.
91.50
```

Formatting changes only how a value is shown: marks still holds 87.456

Quick check

`f"{marks:.2f}"` with `marks = 87.456` shows **87.46**

`f"{percentage:.1%}"` with `percentage = 0.875` shows **87.5%**

Basic String Operations

Strings are **immutable**: methods return a **new** string.

1) Join, repeat, count, index

Tool	Example	Result
+ join	`first_name + " " + last_name`	Asha Patil
* repeat	`"Hi! " * 3`	Hi! Hi! Hi!
len() count	`len(full_name)`	10
[0] first character	`full_name[0]`	A
[-1] last character	`full_name[-1]`	l

Example :

```
first_name = "Asha"
last_name = "Patil"
full_name = first_name + " " + last_name

print(full_name)
print("Hi! " * 3)
print("Length:", len(full_name))
print("First character:", full_name[0])
print("Last character:", full_name[-1])
```

Output :

```
Asha Patil
Hi! Hi! Hi!
Length: 10
First character: A
Last character: l
```

2) Indexing starts at 0



In `city = "Pune"`:

`city[0]` is **P**

`city[-1]` is **e**

The last character is always **-1**.

3) Handy string methods

Method	Example	Result
`.upper()` / `.lower()`	`"Hi".upper()`	"HI"
`.title()`	`"asha patil".title()`	"Asha Patil"
`.split(",")`	`"a,b,c".split(",")`	['a', 'b', 'c']

Methods do not change the original: save the result in a variable

Quick check

`len("Asha Patil")` gives **10**

Mini Project & Common Errors

Combine **input**, **type conversion**, **strip()** and **formatted output** in one program.

1) Student marks report

Example :

```
name = input("Student name: ").strip()
subject = input("Subject: ").strip()
marks = float(input("Marks obtained: "))
total_marks = float(input("Total marks: "))

percentage = marks / total_marks
print("\n--- Student Report ---")
print(f"Name       : {name}")
print(f"Subject    : {subject}")
print(f"Marks      : {marks:.1f} / {total_marks:.1f}")
print(f"Percentage : {percentage:.1%}")
```

Output :

```
Student name:   Asha
Subject: Maths
Marks obtained: 87.5
Total marks: 100

--- Student Report ---
Name       : Asha
Subject    : Maths
Marks      : 87.5 / 100.0
Percentage : 87.5%
```

strip cleaned the spaces around the name, and the number formats shaped the numbers

2) Common errors

Error	When it happens	Example
`NameError`	using a name that is not defined	`print(scroe)`
`ValueError`	text cannot become a number	`int("hello")`
`TypeError`	mixing text and a number	`"Age: " + 19`
`Case matters`	input() is not Input()	`Input("Name: ")`

3) Good practices

- Use `.strip()` for text fields where extra spaces are not wanted.
- Use **f-strings** for readable output.
- Use `float()` for decimal input and `int()` for whole-number input.

Quick check

Which line raises a TypeError? `"Age: " + str(19)`, `"Age: " + 19`, `"Age: " * 2` **"Age: " + 19**

Fix it with `str(19)` or use an f-string.

Practice Tasks & Solutions

Try each task in VS Code first. Then compare with the model solution.

Task 1 Name and city: print a greeting.

Solution :

```
name = input("Name: ").strip()
city = input("City: ").strip()
print(f"Hello {name}, welcome to {city}!")
```

```
Name: Ravi
City: Pune
Hello Ravi, welcome to Pune!
```

Task 2 Two whole numbers: print the sum.

Solution :

```
a = int(input("First number: "))
b = int(input("Second number: "))
print("Sum:", a + b)
```

```
First number: 4
Second number: 5
Sum: 9
```

Task 3 Price x quantity, 2 decimals.

Solution :

```
price = float(input("Price: "))
quantity = int(input("Quantity: "))
print(f"Total: {price * quantity:.2f}")
```

```
Price: 49.5
Quantity: 3
Total: 148.50
```

Task 4 Decimal 0 to 1 as a percentage.

Solution :

```
x = float(input("Decimal between 0 and 1: "))
print(f"{x:.1%}")
```

```
Decimal between 0 and 1: 0.875
87.5%
```

Task 5 Clean a name with spaces.

Solution :

```
name = input("Name: ").strip()
print(name)
```

```
Name:   Asha
Asha
```

Task 6 Full name in title case.

Solution :

```
first = input("First name: ").strip()
last = input("Last name: ").strip()
print(f"{first} {last}".title())
```

```
First name: asha
Last name: patil
Asha Patil
```

Task 7 Length, UPPER and lower.

Solution :

```
text = input("Sentence: ")
print(len(text), text.upper(), text.lower())
```

```
Sentence: Python is fun
13 PYTHON IS FUN python is fun
```

Task 8 Marks as a percentage.

Solution :

```
marks = float(input("Marks obtained: "))
total = float(input("Total marks: "))
print(f"{marks / total:.1%}")
```

```
Marks obtained: 45
Total marks: 60
75.0%
```

Task 9 Split three foods.

Solution :

```
foods = input("Three foods: ").split(",")
print(foods)
```

```
Three foods: pizza,pasta,idli
['pizza', 'pasta', 'idli']
```

Task 10 Add the course name.

Solution :

```
course = input("Course: ").strip()
print(f"Course     : {course}")
```

```
Course: Python
Course     : Python
```

Day 2 Cheat Sheet

Complete revision of Day 2 on one page. **Save and share!**

Input & Strip

- `input()` returns a **str**
- `.strip()` trims both ends
- `.lstrip()` / `.rstrip()`
- Python is case-sensitive

Convert

- `int()` whole numbers
- `float()` decimals
- `int("25.5")` is a ValueError
- `try / except ValueError`

Print() Options

- `sep=` goes between items
- `end=` goes at the end
- `print("a", "b", sep="-")`
- `print("Loading", end="...")`

F-strings

- `f"Hello {name}"`
- `{x:.2f}` two decimals
- `{x:.1%}` percentage
- `{a + b}` expressions

.format() Style

- `"{} scored {}".format(a, b)`
- `"{:.2f}".format(x)`
- works, but f-strings
- are easier to read

String Basics

- `+` join, `*` repeat
- `len()` counts characters
- `s[0]` first, `s[-1]` last
- strings are immutable

String Methods

- `.upper()` / `.lower()`
- `.title()`
- `.split(",")` gives a list
- save the result!

Common Errors

- NameError: undefined name
- ValueError: bad conversion
- TypeError: text + number
- Input() is not input()

Good Practices

- `.strip()` text fields
- f-strings for output
- `float()` for decimals
- `int()` for whole numbers

Key Points To Remember

- ☆ `input()` always returns text; convert it before maths.
- ☆ `.strip()` cleans the edges of text.
- ☆ f-strings with `:.2f` and `:.1%` make output readable.
- ☆ Formatting changes the display, not the stored value.
- ☆ Strings are immutable; methods return a new string.

Next lesson: Day 3 - Strings, Lists and Sets